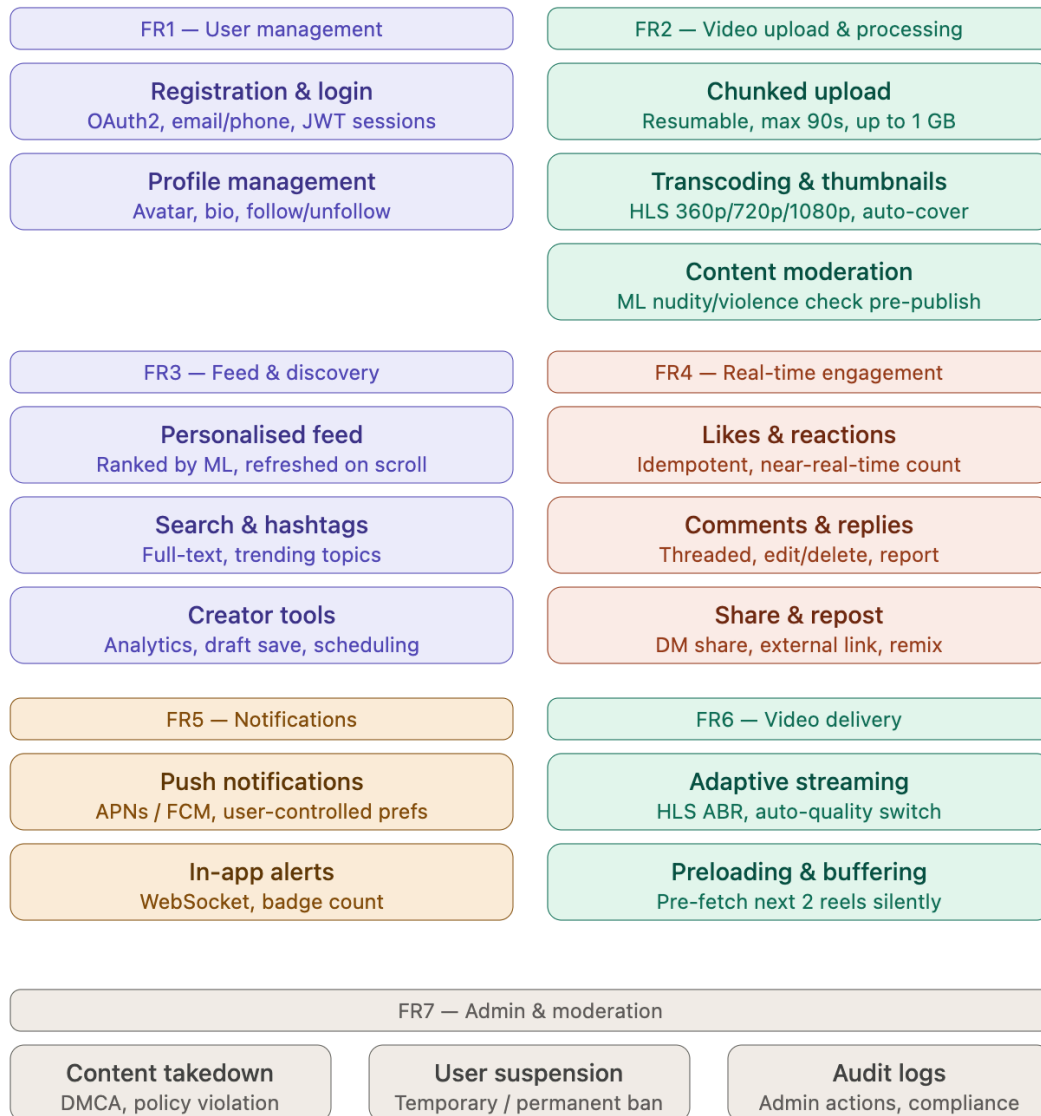


Functional Requirements:	2
Non Functional Requirements	3
Functional requirements — key decisions	3
Non-functional requirements — key decisions	4
Data model:	5
API contract decisions	6
Data model decisions	6
Data Table Details	7
1 — Video upload Sequence	8
2 — Redis caching strategy	9
3 — SQS + SNS event flow with ALB	10

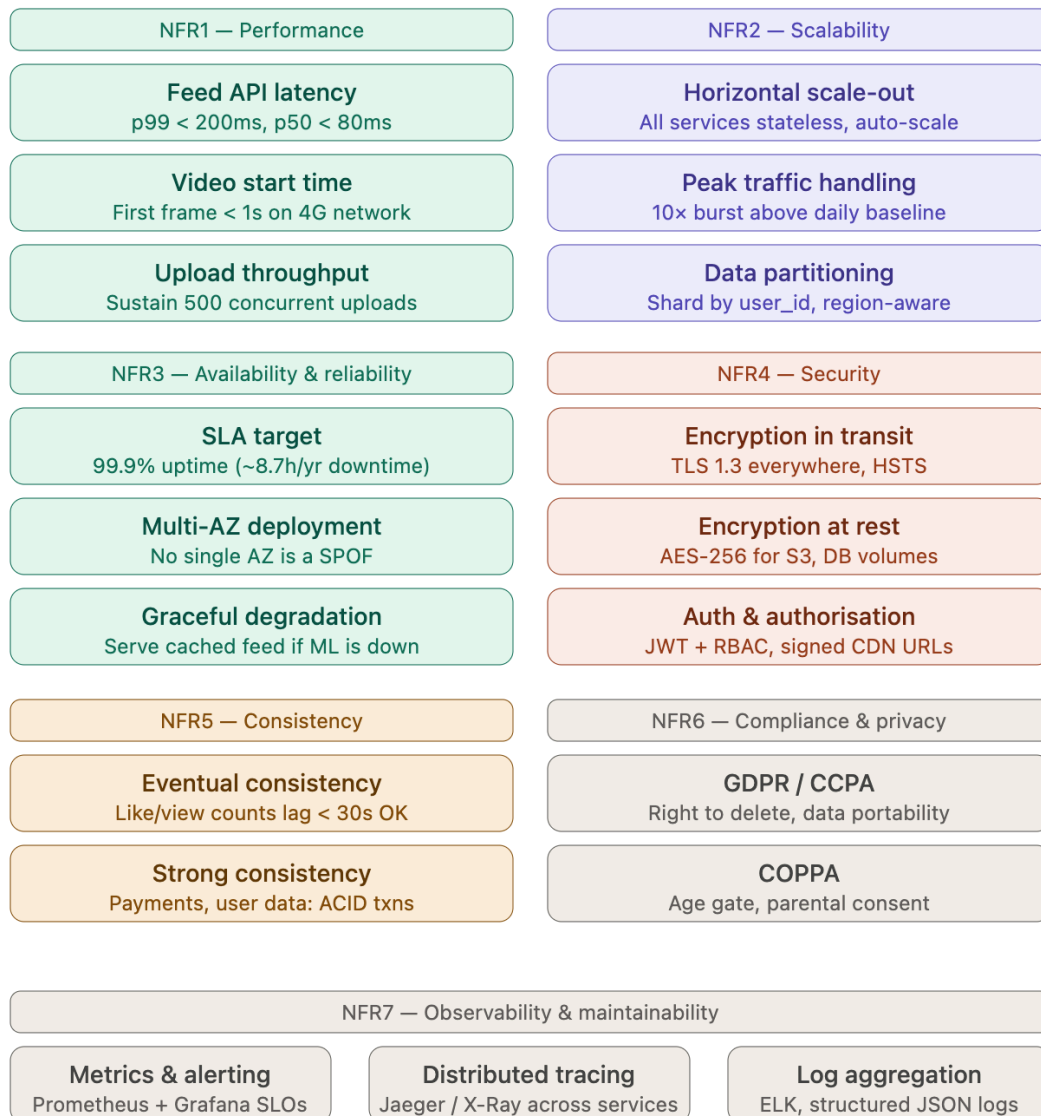
Functional Requirements:

Functional requirements



Non Functional Requirements

Non-functional requirements



Functional requirements — key decisions

FR1 — User management: Use OAuth2 (Google/Apple sign-in) to reduce friction. Sessions managed via short-lived JWT access tokens (15 min) + long-lived refresh tokens (30 days) stored in HttpOnly cookies.

FR2 — Upload & processing: The 90-second video limit and 1 GB cap are practical upper bounds at mid-scale. Content moderation must run before a video becomes publicly visible — use AWS Rekognition or a custom ML model in the transcoding pipeline.

FR3 — Feed & discovery: The feed must support both a "following" tab (social graph) and a "for you" tab (pure ML recommendation). These are separate read paths with different caching strategies.

FR4 — Engagement: Like operations must be idempotent — a user tapping the heart button twice shouldn't create two likes. Use a Redis `SET (SADD reel:likers:{reel_id} {user_id})` rather than a counter increment to guarantee this.

FR5 & FR6 — Notifications and delivery: Pre-fetching the next 2 reels silently is the single biggest UX win for perceived performance. The video player should begin downloading the next reel's first HLS segment before the current one ends.

Non-functional requirements — key decisions

NFR1 — Performance: The p99 < 200ms feed target is achievable only with Redis caching. Without cache, Postgres queries alone can exceed this under load.

NFR2 — Scalability: Every service must be stateless — no server-side sessions, no local file writes. This is what enables horizontal auto-scaling. Stateful things (sessions, file chunks) go into Redis and S3 respectively.

NFR3 — Availability: Multi-AZ is non-negotiable. Graceful degradation means: if the ML ranking service is down, serve a time-sorted feed from the follow graph instead of returning an error.

NFR4 — Security: CDN URLs for video segments must be signed with short expiry (e.g., 1 hour) to prevent hotlinking and content scraping.

NFR5 — Consistency: This is your CAP theorem trade-off. Like counts and view counts can be eventually consistent — users don't notice a 30-second lag. But follow/unfollow and privacy settings must be strongly consistent.

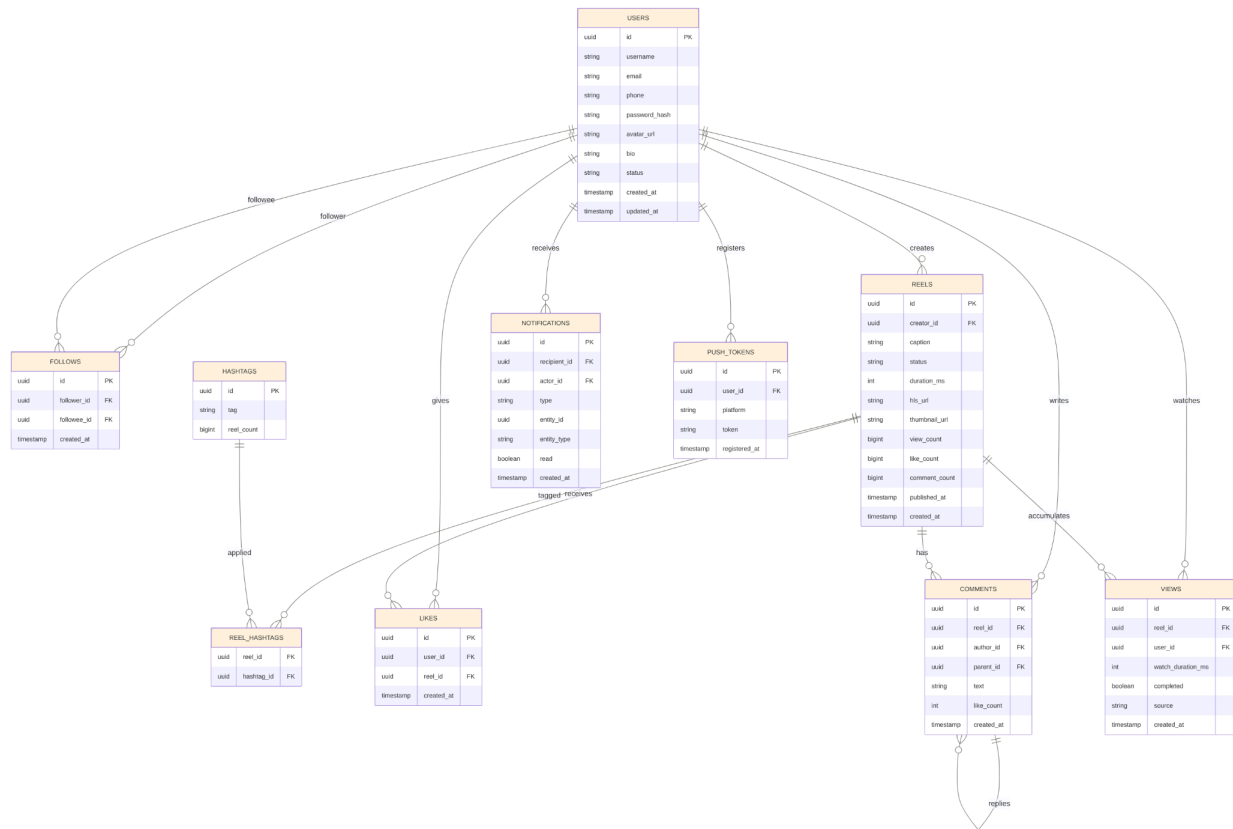
NFR6 — Compliance: GDPR "right to delete" is architecturally tricky — you need a deletion propagation job that removes content from S3, CDN cache, search indexes, and all database tables. Design this from day one, not as an afterthought.

NFR7 — Observability: Define SLOs before launch: feed p99 < 200ms, video start < 1s, uptime > 99.9%. Alert when you breach the SLO budget, not when the service goes completely down.

Here are the full API contracts first, rendered as an interactive reference you can browse by domain:

Auth	Video	Feed	Engagement	Notifications
POST	/v1/auth/register	public	Create account	>
POST	/v1/auth/login	public	Authenticate user	>
POST	/v1/auth/refresh	public	Rotate access token	>
GET	/v1/users/{user_id}/profile	JWT	Get user profile	>

Data model:



PK = primary key FK = foreign key || = one o = zero or many ||-o = one-to-many

API contract decisions

Versioning: All routes are prefixed `/v1/`. When breaking changes are needed, `/v2/` routes are added alongside — never modify existing versioned contracts in-place.

Cursor-based pagination: Every list endpoint uses opaque cursor tokens, not page numbers. Page numbers break when rows are inserted mid-scan. Cursors encode the last-seen item's sort key (e.g., `{created_at, reel_id}` base64-encoded), making pagination stable and stateless.

Upload flow (two-step): `/upload/init` returns pre-signed S3 URLs, then the client uploads chunks directly to S3 — bypassing your servers entirely. Only the completion call hits your backend. This avoids your API servers becoming a bandwidth bottleneck.

Idempotent like toggle: `POST /reels/{id}/like` works as a toggle, returning the new state. This simplifies the client — it never needs to check the current state before acting. The server uses a Redis `SADD/SREM` which is naturally idempotent.

View events are fire-and-forget: `POST /reels/{id}/view` always returns 204. The client doesn't wait for confirmation. Server-side deduplication using a Redis HyperLogLog handles repeat views within a 24-hour window.

Signed HLS URLs: The `hls_url` returned in reel responses includes a short-lived signed CDN token. This prevents hotlinking and ensures access control even after a reel is made private.

Data model decisions

USERS: `status` field handles `active`, `suspended`, `deleted` states — soft deletes only. Hard deletion is a separate async job for GDPR compliance.

FOLLOWS: A simple junction table with a composite unique index on (`follower_id`, `followee_id`). This powers the social graph queries for the following feed.

REELS: `status` can be `uploading`, `processing`, `published`, `failed`, `deleted`. `hls_url` and `thumbnail_url` are only populated once processing completes. Counter columns (`view_count`, `like_count`, `comment_count`) are denormalized here for fast reads — they're updated asynchronously by the SQS consumer flushing Redis counters.

LIKES: Stored in Cassandra (high write throughput), not Postgres. The unique constraint on (`user_id`, `reel_id`) enforces idempotency at the DB level as a safety net behind Redis.

VIEWS: Also Cassandra — this table gets the highest write volume in the system. Partitioned by `reel_id` for fast per-reel analytics, with `user_id` as the clustering key for deduplication queries.

REEL_HASHTAGS: A many-to-many bridge. The `HASHTAGS` table maintains a `reel_count` counter for trending hashtag queries — updated via the same async flush pattern as reel counters.

COMMENTS: The `parent_id` self-referential FK enables one level of threading (comment → replies). Deeper nesting is capped at 1 level to keep queries simple.

NOTIFICATIONS: `entity_id` + `entity_type` form a polymorphic reference — a notification can point to a reel, a comment, or a follow event without separate FK columns per type.

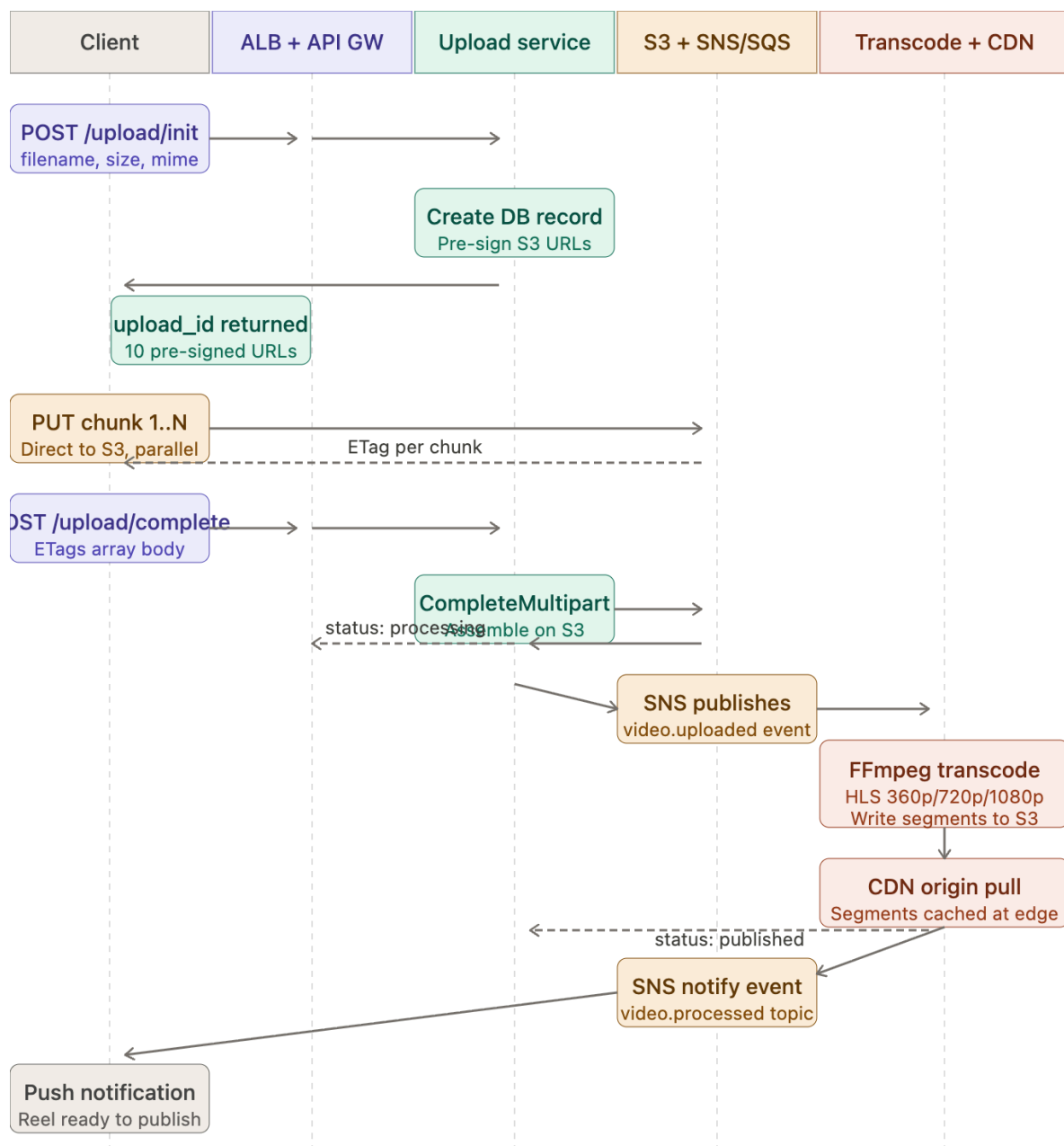
PUSH_TOKENS: Separate table from users — one user can have multiple devices. Tokens are rotated by the client on each app launch and deduplicated by (`user_id`, `platform`, `token`).

Data Table Details

Table	Store	Reason
<code>USERS</code> , <code>REELS</code> , <code>HASHTAGS</code>	PostgreSQL L	ACID, relational joins
<code>FOLLOWS</code> , <code>REEL_HASHTAGS</code>	PostgreSQL L	Graph queries, joins with users
<code>LIKES</code> , <code>VIEWS</code>	Cassandra	Very high write volume
<code>COMMENTS</code>	Cassandra	Wide row per reel, append-heavy
<code>NOTIFICATIONS</code> , <code>PUSH_TOKENS</code>	PostgreSQL L	Low volume, needs transactions
Like/view counters	Redis	Sub-ms reads, async flush to Postgres
Feed cache	Redis	Per-user TTL list of reel IDs

1 — Video upload Sequence

This is the most nuanced sequence in the system. Chunks go directly to S3 — your servers only coordinate, never carry video bytes.



The critical insight here is that video bytes never touch your API servers — the client talks to S3 directly using pre-signed URLs. Your backend only handles small control plane messages. Now for the Redis caching strategy:

2 — Redis caching strategy

Redis key spaces

Colour = data type used

List

Hash

String

Set

HyperLogLog

Feed cache

Key pattern	feed:foryou:{user_id} feed:following:{user_id}
Write	LPUSH reel_ids after ranking — TTL 300s (5 min)
Read	LRANGE 0 19 → serve 20 reels, O(1). Miss → re-rank from DB, LPUSH result

Engagement counters

Key pattern	counters:{reel_id} (Hash with fields: likes, views, comments)
Write	HINCRBY counters:{reel_id} likes 1 — no TTL (always warm)
Flush	SQS consumer runs every 30s → UPDATE reels SET like_count=... in Postgres

Like membership (idempotency)

Key pattern	likers:{reel_id} (Set of user_ids)
Like	SADD likers:{reel_id} {user_id} → returns 1 (new) or 0 (already liked)
Unlike	SREM likers:{reel_id} {user_id} — TTL 24h (evict cold reels)

Session store

Key pattern	session:{refresh_token_hash} → {user_id, issued_at} — TTL 30 days
Usage	SET on login, GET on /auth/refresh, DEL on logout. Rotation: DEL old, SET new

Rate limiting

Key pattern	ratelimit:{user_id}:{endpoint} — TTL = window size
Algorithm	INCR key → check vs limit. If first request: SET with TTL. Fixed window, O(1)

View deduplication

Key pattern	views:hll:{reel_id}:{date} (HyperLogLog) — TTL 25h
Usage	PFADD key {user_id} → 0 if already seen (duplicate). PFCOUNT for unique views

Each key space uses the most efficient Redis data structure for its access pattern. Now the full event flow with ALB + SQS/SNS replacing Kafka:

3 — SQS + SNS event flow with ALB

